

PCS3225 - Sistemas Digitais II: Exercício da Aula 8 - Síntese de Hardware I (Exemplo de Solução)

Antonio Vieira da Silva Neto e Gabriel Pereira de Carvalho

Data: 31/08/2026

O objetivo deste exercício é especificar e implementar uma modificação do circuito 'Contador de Uns customizado' apresentado no material. Para tanto, deve-se desenvolver uma especificação RTL (*Register-Transfer Level*), identificar os elementos do fluxo de dados, elaborar o diagrama de transição de estados da Unidade de Controle, desenvolver o Sistema Digital em abordagem *bottom-up*, especificar casos de teste e realizar a verificação funcional do circuito. A implementação em VHDL deve incluir a codificação de todos os componentes, do módulo do fluxo de dados, do módulo da unidade de controle e do sistema digital completo. A verificação funcional de funcionamento deve ser realizada com a implementação dos casos de teste, a criação de um *testbench* e a simulação com GHDL/GTKWave, EDAPlayground ou Quartus/ModelSim.

Este exercício visa praticar um projeto completo e iterativo de um sistema digital, incluindo sua concepção, seu projeto, sua implementação em uma HDL, sua verificação funcional e como modificações podem ser incluídas no projeto.

Agradecimentos: As soluções apresentadas aqui utilizam como base as submissões de alunos da disciplina em anos anteriores. Dessa forma, são dirigidos agradecimentos aos alunos André Bessa da Franca Saliba, Arthur Wunder Damásio, Henrique Feldman Falcão, João Vítor Guimarães Santos Rocha, Pedro Monteiro Beraldo, Thiago Franco Medeiros, Daniel Brunoro Krohling, Felipe Lima de Assis Pereira (monitor da disciplina em 2026), Gabriel Carillo, Henrique Ponce de Leon Camargos, Leonardo Kenji Kiukawa e Vitor Katzenellenbogen Teixeira.

Textos em [azul](#) nesse tutorial possuem *hyperlinks* que podem ser clicados para levar à página referenciada.

Todo o material das soluções apresentadas aqui e o código VHDL do projeto estão disponíveis [num repositório Github criado para esse exercício](#).

Enunciado

Partindo do projeto do *contador de uns customizado*, apresentado nos **slides complementares da Aula 08 (Síntese de Hardware I)** e cujo código VHDL está disponível na **Aula 00 (Revisão de Projeto de Sistemas Digitais)**, deve-se especificar, implementar e verificar uma modificação. Ela consiste em um segundo modo de funcionamento, selecionado por meio de um sinal denominado 'modo', que realize a função de 'Contador de Duplos Uns'. As demais interfaces de entrada e saída não devem ser modificadas.

As regras de funcionamento do circuito modificado são as seguintes:

1. **Seleção entre Modos:** Quando a entrada *modo* é igual a '0', o circuito deve operar com o Contador de Uns já existente; caso contrário, ele deve operar com o Contador de Duplos Uns a ser incluído. O circuito NÃO DEVE permitir que mudanças na entrada 'modo' alterem o funcionamento do circuito se a entrada 'start' já tiver sido acionada;
2. **Contador de Duplos Uns:** Sempre que houver dois uns seguidos na entrada *inport*, a contagem deve ser incrementada em uma unidade. Exemplos: Entrada 000000000110000: Saída 1; Entrada 000011001100000: Saída 2; Entrada 010101010101010: Saída 0; Entrada 111000011100000: Saída 4; Entrada 111111111111111: Saída 14;
3. **Interfaces do Circuito:** Com exceção da entrada *modo*, a ser incluída na entidade do sistema como *bit*, todas as demais interfaces de entrada e saída do contador de uns 'customizado' devem ser mantidas inalteradas.

Nesta parte da atividade, as seguintes tarefas devem ser realizadas:

1. Descreva o funcionamento do circuito modificado utilizando as seguintes especificações RTL de alto nível de abstração: pseudocódigo, CDFG (*Control Data Flow Graph*) e FSM (*Finite State Machine With Data*);

Pseudocódigo: No pseudocódigo para o circuito modificado, uma possibilidade é adicionar no início da operação um desvio condicional baseado no valor do sinal de entrada *modo*.

Se *modo* = 0, então temos o mesmo funcionamento do contador de uns customizado. Porém se *modo* = 1, então a cada iteração temos que verificar se os dois bits menos significativos do registrador de deslocamento são ambos iguais a 1 para incrementar a contagem.

```
while (1){
    while ( Start == 0);
    Done = 0;
    Data = Input;
    Ocount = 0;
    if ( modo == 0){
        while (Data > 0){
            Ocount += Data(0) & 1;
            Data >>= 1;
        }
    }else{ // modo = 1
        while (Data > 0){
            Ocount += Data(0) & Data(1);
            Data >>= 1;
        }
    }
    Output = Ocount;
    Done = 1;
}
```

Observe que o pseudocódigo para ambos os casos *modo* = 0

e $\text{modo} = 1$ são bastante parecidos. Em um caso calculamos $\text{Data}(0) \text{ AND } 1$ e no outro $\text{Data}(0) \text{ AND } \text{Data}(1)$.

Nosso objetivo nessas soluções é buscar minimizar o impacto das mudanças feitas ao código fornecido para o exercício. Com esse objetivo em mente, podemos simplificar o pseudocódigo introduzindo uma variável adicional *Mask* que assume o valor desejado em cada um dos casos. Ou seja, queremos

$$\text{Mask} = \begin{cases} 1 & \text{se modo} = 0 \\ \text{Data}(1) & \text{se modo} = 1 \end{cases} \quad (1)$$

Note que *Mask* é igual a zero se, e somente se, $\text{modo}=1$ e $\text{Data}(1)=0$. Portanto, temos

$$\overline{\text{Mask}} = \text{modo AND } \overline{\text{Data}(1)}$$

e, aplicando as leis de De Morgan, temos

$$\text{Mask} = \overline{\text{modo}} \text{ OR } \text{Data}(1)$$

com isso chegamos ao seguinte pseudocódigo

```
while (1){
    while ( Start == 0);
    Done = 0;
    Data = Input;
    Ocount = 0;
    Mask = 1;
    Reg_modo = modo; // constante durante a operação

    while ( Data > 0){
        Mask = (not Reg_modo) || Data(1);
        Ocount += Data(0) & Mask;
        Data >>= 1;
    }
    Output = Ocount;
    Done = 1;
}
```

Observe que no código VHDL fornecido, a variável *Ocount* é representada pela saída de um contador de 4 bits. No nosso pseudocódigo, o valor binário $\text{Data}(0) \& \text{Mask}$ pode ser utilizado como o valor de enable desse contador.

CDFG (feito com o drawio):

O CDFG apresentado é bastante similar ao diagrama para o contador de uns original fornecido para o exercício. A grande diferença está no bloco de cálculo onde processamos os dois bits menos significativos do sinal Data para cada iteração que agora envolve duas novas variáveis Mask e Reg_mod.

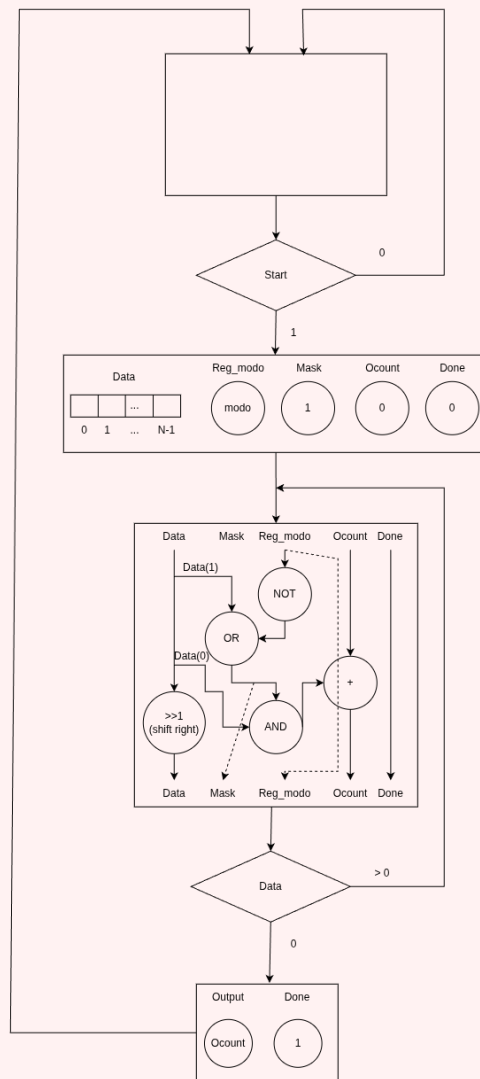


Figura 1: CDFG para sistema proposto

FSMD (feito com o drawio):

Os desvios condicionais no CDFG sugerem uma divisão bastante natural da sequência de cálculos propostos em estados para o projeto da unidade de controle. Ao desenhar a FSMD, estamos propondo uma máquina de estados de Moore para a unidade de controle e dividindo as operações do CDFG entre os estados propostos.

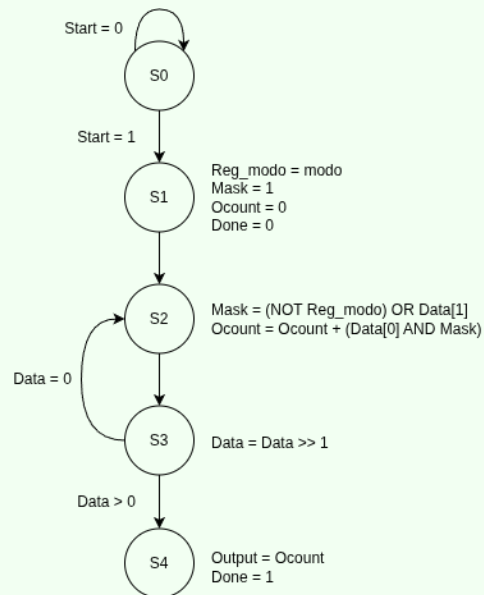


Figura 2: FDSM para sistema proposto

2. Elabore, a partir do item anterior, a especificação RTL do sistema em diagrama ASM (*Algorithmic State Machine*);

Diagrama ASM:

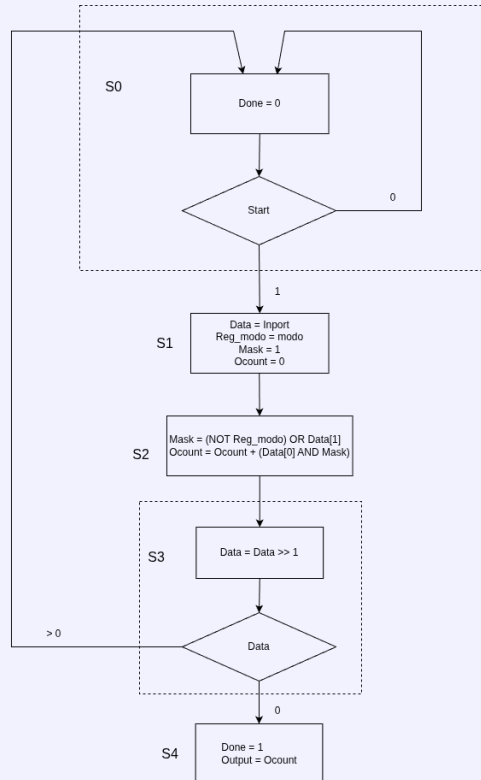


Figura 3: ASM para sistema proposto

3. Identifique se há necessidade de novos componentes para o Fluxo de Dados e, em caso afirmativo, indique quais são eles;

Para garantir que mudanças no sinal de entrada modo durante o processamento do vetor não afetem o comportamento do circuito, é importante adicionar ao fluxo de dados um registrador `Reg_mod` representando a variável de mesmo nome que definimos na etapa do pseudocódigo.

4. Identifique se há necessidade de novos sinais de condição e de controle e, em caso afirmativo, descreva-os;

Para controlar a operação do registrador `reg_mod`, adicionamos uma saída `reg_mod_enable` à unidade de controle. Utilizando a técnica de **clock gating**, esse sinal permite a escrita de um novo valor de modo no registrador apenas no estado inicial da máquina de estados, quando a entrada `start` ainda não foi ativada. É necessário também modificar o registrador de deslocamento `deslocador_n` para obter na saída os atuais dois bits menos significativos, `data0` e `data1` do vetor de entrada sendo processado.

5. Especifique casos de teste relevantes para exercitar o funcionamento do circuito modificado, de forma a garantir não só a correção do 'contador de duplos uns' e da seleção do modo de operação, mas também a **não regressão** do contador de uns original;

O testbench que foi fornecido no projeto base realiza testes exaustivos cobrindo todos os possíveis valores da entrada e comparando a saída obtida com uma função de referência capaz de avaliar a corretude do projeto sem necessidade de construir um grande vetor de testes com todos os valores de saída pré-calculados.

A abordagem proposta aqui é modificar o testbench para realizar testes exaustivos para `modo = 0` (não regressão) e `modo = 1`.

Teste	Objetivo	inport	modo	Justificativa
1	Reset Global	-	-	Verifica se o sinal de reset zera as saídas e estado inicial.
2	Não regressão (<code>modo=0</code>)	$0, 1, \dots, 2^{15} - 1$	0	Teste exaustivo do contador de uns.
3	Não regressão (<code>modo=0</code>)	$0, 1, \dots, 2^{15} - 1$	1	Teste exaustivo do contador de duplos uns.
4	Imutabilidade do Modo	000000000110000	$0 \rightarrow 1$	Caso de teste que muda modo de 0 para 1 após <code>start = 1</code> .

6. Implemente o novo circuito em VHDL. DICA: Siga uma abordagem *bottom-up*, começando pelos seus componentes e avançando até a entidade topo;

Primeiramente, adicionamos uma entidade `reg.vhd` para implementar o registrador da variável `Reg_mod0`. Então adicionamos ao fluxo de dados esse registrador e as fórmulas mostradas no pseudocódigo para calcular o enable do contador para a variável `0count`. A máquina de estados foi modificada para a estrutura reduzida com cinco estados proposta no diagrama da FSM.

Para analisar em detalhe o código proposto como solução, por favor [conferir o repositório Github criado para esse exercício](#).

7. Codifique o *testbench* em VHDL usando uma das formas de codificação da Aula 2: programático, vetor de testes ou arquivo externo;

Assim como no testbench do projeto original, utilizamos o **método programático** e comandos `assert` para que, quando um erro for detectado na saída de um caso de teste, simulação seja abortada e se imprima uma mensagem de erro. Como estamos verificando de forma exaustiva um número muito grande de testes, seria difícil verificar manualmente a saída de cada uma das formas de onda. Assim, essa abordagem permite que, quando um erro é detectado, temos o instante de tempo para analisar sua causa nas formas de onda.

Para analisar o testbench proposto como solução, por favor [conferir o repositório Github criado para esse exercício](#).

8. Simule os casos de teste. Os resultados podem vir das formas de onda geradas ou pelas mensagens via comandos de `assert` e `report`;

Executar o testbench sem visualizar nenhuma das mensagens de erro associadas aos comandos `assert` com a mensagem de validação ao final é um sinal de sucesso na verificação funcional do projeto.

```
gabriel$ ./sim_onescounter.sh
../tb/onescounter_tb.vhd:72:5:@0ms:(assertion note): BOT
../tb/onescounter_tb.vhd:121:4:@21626940ns:(assertion note): Todos os testes ok!
../tb/onescounter_tb.vhd:122:4:@21626940ns:(assertion note): EOT
```

Figura 4: Testbench do projeto passa em todos os casos de teste

9. Analise os resultados da simulação de forma criteriosa. Com que cobertura sua simulação garante que o contador funciona corretamente nos dois modos de operação (uns e duplos uns)?

É importante comentar aqui que nem sempre a abordagem de verificação exaustiva proposta nesse exercício pode ser aplicada numa simulação. Em projetos de alta complexidade, é inviável querer testar todos os valores possíveis da entrada.

Na área de verificação de sistemas digitais, existem duas métricas muito importantes usadas para determinar a qualidade de um plano de testes: *cobertura de código*(*code coverage*) e *cobertura funcional*(*functional coverage*).

Para uma cobertura de código de 100%, a simulação precisa executar todas as linhas de código RTL do projeto. Ou seja, o objetivo de se obter uma alta cobertura de código é garantir que o código que foi implementado está sendo executado pela simulação, mas isso não quer dizer que esse código está cobrindo toda a especificação funcional do projeto.

Para uma cobertura funcional de 100%, o funcionamento de todas as *features* presentes na especificação funcional do projeto precisa ser verificado. Se trata de uma métrica completamente independente da forma em que o projeto foi implementado, mas é mais difícil de ser quantificada uma vez que requer uma ferramenta capaz de comparar os resultados da simulação com um modelo formal da especificação do projeto.

Para facilitar a tarefa de verificação em projetos de alta complexidade, existem metodologias de verificação que definem formas específicas de construir o plano de testes e bibliotecas para suportar essa construção e facilitar a coleta de métricas como a cobertura de código e a cobertura funcional. O padrão absoluto dentro da indústria é a *Universal Verification Methodology (UVM)*.